

Team Name

The Matrix

SmartServe

See the portion. Know the choice.

Project Documentation (Updated with Repository Features)

Team Members

Thanusha K R

Rakshitha S

VijayKumar Mantur

SriHarsha N

QR-first Restaurant Web Platform — Table Verification, 3D/AR Dish Preview, Customization and a Real-time Customer-Kitchen-Server-Manager Workflow

Live app: smart-serve-kb21.vercel.app

Source: github.com/thanushakr/Smart_serve

What Is New in This Version

This update aligns the original documentation with the current state of the SmartServe repository (main branch, 53 commits). The core idea is unchanged; the implementation has matured from a planned architecture into a deployed, working prototype.

Area	Original plan	Current implementation
Backend / data	Node.js + Express, PostgreSQL + Prisma, Socket.IO	Firebase: Authentication and Cloud Firestore with live subscriptions; Firebase Storage client
Frontend	React + TypeScript + Vite, Tailwind, shadcn/ui	React 19, TypeScript, Vite 8, Tailwind CSS 4, React Router 7, Zustand 5, TanStack Query, Lucide React
Order lifecycle	new, preparing, ready, completed	NEW, ACCEPTED, PREPARING, READY, COMPLETED, synced in real time across all four roles
Customer tracking	Intentionally removed; success screen only	Live order-status screen at /order-status/:orderId, plus order confirmation
Table entry	Common QR + 4-digit code, hashed	Table-aware QR link (/verify?table=table-12); code checked against an active Firestore table; locally persisted session
Staff access	Server-side RBAC (planned)	Firebase email/password login, Firestore staff profiles, role-protected routes for manager, kitchen and server
Manager tools	Overview, menu, tables, staff, analytics	Seven manager routes including Orders, Menu, Tables (QR/code controls), Staff, Analytics and a prototype 3D Model Studio
Menu decisions	3D, scale, allergens, combos	Search, category filters, nutrition, portions, spice levels, add-ons, favorites, combos and cart totals
3D / AR	model-viewer + Three.js	Google model-viewer with GLB assets; WebXR, Scene Viewer and Quick Look hand-off; normal 3D view as fallback
Deployment	Vercel + Render/Railway	Single Vercel deployment with SPA route rewrite (vercel.json)

Note: items from the original design that depend on a custom backend or the OpenAI API (hunger-fit guidance, AI safe swaps, multi-diner shared cart) are not part of the current repository and are carried forward as roadmap items in Section 9.

1. Empathizing

Conventional restaurant menus communicate a dish only through its price and a static photograph. Neither conveys the true physical quantity of food being ordered, so diners struggle to judge whether a dish is enough for one, enough to share, or too small for a hungry group. This gap between what a menu shows and what a diner needs to decide is the core pain point SmartServe was designed around.

Observing the ordering journey from the customer's perspective reveals recurring frustrations:

- Diners over-order out of caution, causing food waste and unnecessary spending.
- Diners under-order out of caution, causing hunger and re-ordering mid-meal.
- Groups sharing a table find it hard to jointly estimate how much food the table needs.
- Diners with allergies or dietary restrictions cannot easily verify ingredients at the point of ordering.
- After ordering, diners have no visibility of whether the kitchen has accepted or started their food.
- Restaurants face unpredictable demand and wastage because ordering is driven by guesswork, and staff lose time relaying orders between front-of-house and kitchen.

These observations form the foundation for SmartServe: an ordering experience that restores the missing sense of scale, quantity and ingredient clarity, and keeps diner and staff in sync through one live workflow.

2. Literature Survey

Most QR-based digital menus simply digitize the printed menu: a name, a price and a photo, with portion estimation left to the customer. Before finalizing the architecture, the team surveyed technologies able to close this gap.

- **3D and AR on the web:** Google's model-viewer web component provides interactive GLB viewing with WebXR, Android Scene Viewer and iOS Quick Look support and a real-world scale mode. Three.js (WebGL/WebGPU, WebXR tooling) was surveyed for custom scenes. WebXR requires HTTPS.
- **Real-time data:** Socket.IO was surveyed for bidirectional live updates. During implementation the team adopted Cloud Firestore, whose real-time listeners deliver the same live-queue behaviour without running a separate socket server.
- **Authentication and roles:** Firebase Authentication with Firestore-held staff profiles was chosen for role-based staff access with minimal infrastructure.
- **Relational option:** PostgreSQL with JSONB and Prisma remains the documented path if the platform later needs strict relational multi-tenancy.
- **AI layer:** The OpenAI API (server-side SDK, structured JSON-schema outputs) was surveyed for hunger-fit and safe-swap guidance, and is retained as a future integration.
- **Image-to-3D:** Meshy and Tripo expose APIs that turn photographs into GLB/USDZ assets and are the intended providers for automated model generation.
- **Frontend tooling:** React documentation deprecates Create React App, so the project is scaffolded with Vite. React Router, Zustand and TanStack Query handle routing, state and data caching.

3. Problem Statement

Restaurant menus, whether printed or digitized through a QR code, communicate price and a static image but not the physical quantity, portion suitability or ingredient and allergen information of a dish at the point of ordering. This missing decision layer between browsing and ordering leads to over-ordering and waste, under-ordering and repeat orders, difficulty in group planning, and risk for allergy-sensitive diners. There is a need for a restaurant platform that gives diners an accurate, real-world sense of portion and ingredient information before they order, while giving staff the tools to manage tables, kitchen queues and sales from the same shared, real-time data.

4. Ideation

Ideation started from the missing sense of scale and ingredient clarity and worked outward to a complete ordering and operations experience.

Customer-facing features

- Interactive 3D dish view with rotate, zoom and fullscreen controls.
- Real-world scale reference (plate, can, hand) with serving estimate, spice level and texture context.
- Dish detail with image, price, rating, preparation time, nutrition, ingredients, allergens, portion sizes, spice levels and add-ons.
- Search, category filters, favorites and a Recommended section.
- Combo offers presented as bundled savings.
- AR placement of the dish on the physical table where the device supports it.
- Cart with totals, order confirmation and a live order-status screen.
- Secure table entry through a QR link and table verification code.

Staff-facing features

- Kitchen: live queue with filters, item and customization details, accept/reject, preparation state and ready hand-off.
- Server: table overview, ready-order notifications, active order details, serving workflow and payment overview.
- Manager: operations overview, orders, menu, tables (QR/code controls), staff records, analytics and a prototype 3D Model Studio.
- Role-protected routes behind Firebase Authentication.

Navigation was consolidated to three primary tabs (Menu, Combos, Cart), with profile, favorites and secondary actions placed behind icons to keep the main experience simple.

5. Proposed Solution

SmartServe is a QR-first restaurant web platform that turns a conventional menu into an interactive, portion-aware ordering experience. Each diner inspects a dish as a realistic 3D model, compares it with familiar reference objects, reviews ingredients, allergens and nutrition, customizes portion, spice level and add-ons, and can preview the dish on their own table through AR before ordering.

End-to-end customer flow

Scan table QR > Verify table code > Browse menu > Customize dish > Preview in 3D or AR > Add to cart > Place order > Track live status

The QR link carries the table identity (for example /verify?table=table-12). The diner enters the verification code displayed at the table, the app checks it against an active table record in Firestore, and a dining session is created and persisted locally so the diner never picks a table number manually.

One order, four connected experiences

Customer places order (NEW) > Cloud Firestore > Kitchen dashboard (ACCEPTED / PREPARING / READY) > Cloud Firestore > Server dashboard (COMPLETED) > Customer status screen and Manager analytics

Experience	What it provides
Customer	Table verification, live menu, search and categories, dish customization, 3D/AR, combos, cart, checkout and live order tracking
Kitchen	Live queue filters, item and customization details, accept/reject controls, preparation state and ready hand-off
Server	Table overview, ready-order notifications, active order details, serving workflow and payment overview
Manager	Operations overview, order and menu management, table QR/code controls, staff records, analytics and a prototype 3D Model Studio

6. Model Development / Prototype

SmartServe is built as a single-page React application backed by Firebase. Customer, Kitchen, Server and Manager experiences share one codebase and one live data source, so there is no reconciliation between separate systems.

Architecture

React 19 + TypeScript + Tailwind CSS 4 (Vite 8) > Zustand (session, cart, viewer state) + TanStack Query > Firebase Authentication + Cloud Firestore (+ Storage client) > model-viewer, GLB assets in /public/models > Vercel hosting

Realistic 3D and AR

Supported dishes ship with GLB models rendered by Google model-viewer, which can be rotated and zoomed in the browser. On compatible phones the same viewer hands off to WebXR, Android Scene Viewer or iOS Quick Look for an AR preview. AR depends on device, browser, model format and HTTPS; where AR is unavailable the standard 3D viewer remains so ordering is never blocked. Routes: /dish/:id, /dish/:id/3d and /dish/:id/ar.

Portion and scale awareness

Each dish carries portion options, serving context, spice levels, nutrition and ingredient details, and is shown against familiar reference objects, so the diner judges quantity visually instead of guessing from a photo.

Secure table entry and sessions

A table-aware QR link plus a verification code ties each customer session to a physical table. The code is validated against an active table document, and the resulting session is persisted locally so a refresh does not lose the cart.

Real-time order lifecycle

Orders are written to Firestore with their session, table, line items, totals, status and timestamps. Firestore subscriptions move an order through NEW, ACCEPTED, PREPARING, READY and COMPLETED, updating the kitchen, server, manager and customer screens without a manual refresh.

Staff dashboards and access control

- **Kitchen:** live queue with filters, customization details, accept/reject and state progression to ready.
- **Server:** table status overview, ready-order notifications, active order details, serving workflow and payment overview.
- **Manager:** overview, orders, menu, tables (QR and verification-code controls), staff, analytics, and the 3D Model Studio.
- **Access:** Firebase email/password login; Firestore staff profile (restaurant, active flag, role) decides which workspace the user may open.

3D Model Studio (prototype)

The manager's Model Studio demonstrates the intended upload-to-3D workflow. It currently simulates the generation step by associating an existing GLB path with an uploaded preview; a production image-to-3D service (Meshy or Tripo) is a planned integration.

Application Routes

Area	Route
Table verification	/verify
Customer menu and combos	/menu, /combos
Dish, 3D and AR	/dish/:id, /dish/:id/3d, /dish/:id/ar
Cart and order tracking	/cart, /order-confirmation, /order-status/:orderId
Staff login	/staff/login
Kitchen and server	/kitchen, /server
Manager workspace	/manager, /manager/orders, /manager/menu, /manager/tables, /manager/staff, /manager/analytics, /manager/models

Data Model (Firestore Collections)

Collection	Purpose
restaurants	Restaurant identity and seed marker
tables	Table number, active state, verification code and operational status
menuItems	Dish content, pricing, availability, images, nutrition and model URLs
orders	Customer session, table, line items, totals, status and timestamps
staff	Authenticated staff profile, restaurant, active flag and role

Technology Stack

Layer	Technology
UI	React 19, TypeScript, Tailwind CSS 4, Lucide React
Routing and state	React Router 7, Zustand 5, TanStack Query
Cloud	Firebase Authentication, Cloud Firestore, Firebase Storage client
3D and AR	Google model-viewer, GLB/glTF, WebXR, Scene Viewer, Quick Look
Tooling and hosting	Vite 8, Oxlint, Vercel (SPA rewrite in vercel.json)

Repository Structure

```
src/components/{auth, common, customer, kitchen, manager, server}
src/{data, services, store, types}
public/{images, models} docs/screenshots vercel.json vite.config.ts
```

Implementation Status

Capability	Status
Table verification, menu, customization, cart, checkout, live tracking	Implemented
Firestore real-time order workflow and role-based routing	Implemented
Kitchen, Server and Manager dashboards	Implemented
3D viewing with GLB assets; AR where device supports it	Implemented (device-dependent AR)
Recommended dishes	Curated menu flags; no trained model

Capability	Status
3D Model Studio generation	Simulated; production image-to-3D planned
Staff records in manager UI	Stored locally; auth accounts and profiles provisioned separately
Restaurant ID	Demo restaurant spice-garden; multi-tenancy planned

7. Impacts and Benefits

For Diners

- A realistic sense of portion size before ordering, reducing over- and under-ordering.
- Ingredients, allergens, nutrition, portions and spice levels visible at the point of decision.
- Customization of portion, spice and add-ons before the dish reaches the cart.
- Live order tracking after checkout, so there is no uncertainty about the food.
- The ability to see a dish on their own table through AR before committing.

For Restaurants

- Less food waste and more predictable demand as customers order with clearer quantity information.
- Real-time kitchen queue and server notifications that reduce miscommunication between front-of-house and kitchen.
- A manager workspace for orders, menu, tables, staff and analytics, plus 3D asset management.
- Table QR and verification-code controls that tie orders to the right table and reduce misuse.

Taken together, the benefit is a single data thread, from a diner's 3D/AR portion decision through the kitchen queue and server hand-off to the manager's analytics, rather than disconnected ordering and operations tools.

8. Feasibility

Technical Feasibility

The working prototype demonstrates feasibility: it is deployed on Vercel, uses documented and actively maintained technology (React, Vite, Firebase, model-viewer, WebXR), and runs real-time order flow across four role-based experiences. Choosing Firebase removed the need to build and host a separate API and socket layer.

Economic Feasibility

Managed, pay-as-you-go services (Firebase, Vercel) keep initial cost low and deployment fast. Starting with a small set of prepared GLB models limits early cost while demonstrating the full experience; paid image-to-3D and AI services are deferred until the core experience is validated.

Operational Feasibility

Clear roles (Kitchen, Server, Manager) each have a scoped workspace guarded by authentication and role checks. Because all views share the same Firestore data, no reconciliation between ordering, kitchen and table systems is needed.

Practical Feasibility

The project was built within a hackathon setting and has progressed to 53 commits with a live deployment. Known constraints (Firestore security rules to be hardened before real data, AR device dependence, simulated model generation) are documented so the team can plan the next phases with clear expectations.

9. Conclusion / Future Scope

SmartServe reframes restaurant ordering around the one piece of information a conventional menu cannot communicate: how much food a dish actually is. By combining 3D models, scale awareness, AR placement, rich dish information and a real-time operations workflow, it gives diners a genuine decision layer between browsing and ordering and gives restaurants one system spanning customer, kitchen, server and manager. 3D and AR remain the differentiating experience layer on top of a coherent, secure, table-aware ordering platform.

Roadmap

Phase	Scope
Delivered (current)	Table verification, customizable menu, 3D/AR viewing, combos, cart and checkout, live order tracking, Kitchen / Server / Manager dashboards, role-based staff login, Vercel deployment
Next	Production image-to-3D generation with optimized model delivery; server-managed staff provisioning; hardened Firestore security rules; expanded automated tests and code splitting
Intelligence	Data-driven personalized recommendations; AI hunger-fit guidance and allergen-aware safe swaps; shared multi-diner table cart with per-person attribution
Engagement	Customer accounts and order history; push notifications for order updates; Progressive Web App with offline-friendly menu
Scale	Payment gateway and verified payment state; multi-restaurant tenancy; richer analytics and business intelligence; voice and multilingual assistant; health-app integrations

App Screens

Reference screenshots for this section are in the repository under docs/screenshots. Insert them here when finalizing the submission:

Screen	Repository file
Customer menu	docs/screenshots/customer-menu.png
Table verification	docs/screenshots/table-verification.png
Dish details	docs/screenshots/dish-details.png
Interactive 3D viewer	docs/screenshots/interactive-3d-viewer.png
Staff login	docs/screenshots/staff-login.png